
Computational Arbitrage in AI Model Markets

Ricardo Olmedo¹ Bernhard Schölkopf¹ Moritz Hardt¹

Abstract

Consider a market of competing model providers selling query access to models with varying costs and capabilities. Customers submit problem instances and are willing to pay up to a budget for a verifiable solution. An arbitrageur efficiently allocates inference budget across providers to undercut the market, thus creating a competitive offering with no model-development risk. In this work, we initiate the study of arbitrage in AI model markets, empirically demonstrating the viability of arbitrage and illustrating its economic consequences. We conduct an in-depth case study of SWE-bench GitHub issue resolution using two representative models, GPT-5 mini and DeepSeek v3.2. In this verifiable domain, simple arbitrage strategies generate net profit margins of up to 40%. Robust arbitrage strategies that generalize across different domains remain profitable. Distillation further creates strong arbitrage opportunities, potentially at the expense of the teacher model’s revenue. Multiple competing arbitrageurs drive down consumer prices, reducing the marginal revenue of model providers. At the same time, arbitrage reduces market segmentation and facilitates market entry for smaller model providers by enabling earlier revenue capture. Our results suggest that arbitrage can be a powerful force in AI model markets with implications for model development, distillation, and deployment.[†]

1. Introduction

Industry experts call it a “Cambrian explosion”—the rapid growth of competing AI models in the marketplace (Chernova, 2025). As of January 2026, Open Router provides API access to more than 600 different models, while Microsoft

¹Max Planck Institute for Intelligent Systems, Tübingen. Correspondence to: Ricardo Olmedo <rdo@tue.mpg.de>.

Proceedings of the 43rd International Conference on Machine Learning, Seoul, South Korea. PMLR 306, 2026. Copyright 2026 by the author(s).

[†]Code, data, and models available at <https://github.com/RicardoDominguez/computational-arbitrage>

Foundry lists more than 11000 models. Customers face myriad model choices of varying costs and capabilities from all kinds of providers. From the customer’s perspective, massive API markets are therefore particularly appealing for workflows with verifiable solutions, such as software passing a test suite. When solutions are verifiable, customers can choose freely between different options without the need for familiarity or trust in the model provider.

At the same time, the complex fragmentation of the AI API market creates potential for *arbitrage*. The same level of performance implicitly trades at different prices across different API endpoints. Solving half of all SWE-bench problems costs roughly \$10 with GPT-5 mini and \$20 with DeepSeek v3.2. However, DeepSeek typically solves harder problems in fewer attempts than GPT-5. At a higher target of a 75% SWE-bench solve rate, DeepSeek is the cheaper model.

In this work, we study how an arbitrageur may exploit such cost asymmetries for profit. A simple arbitrage strategy first queries GPT-5 mini some number of times before switching over to DeepSeek. Easier problems are thus cheaply solved by GPT-5 mini, while more difficult ones are ultimately handled by DeepSeek. By choosing a good switch-over point, the arbitrageur can purchase a target level of performance at a lower cost than either model alone. This cost advantage is in turn a profit opportunity (see Figure 1). The strategy requires no up-front investment and is formally risk-free when customers commit to a no-refund budget. At worst, the arbitrageur fails to create a competitive product and ends up with zero profit, but no losses.

Arbitrage is fundamental to financial markets, yet we lack a corresponding understanding of its computational counterpart in the context of AI model markets. Initiating the study of computational arbitrage in AI model markets, we empirically demonstrate the viability of arbitrage in a realistic setting and illustrate its potential economic consequences. Our results suggest that arbitrage in AI model markets may have powerful implications for model development, distillation, and deployment.

In more detail, our contributions are:

- **We formalize the concept of computational arbitrage in AI model markets.** Arbitrage opportunities

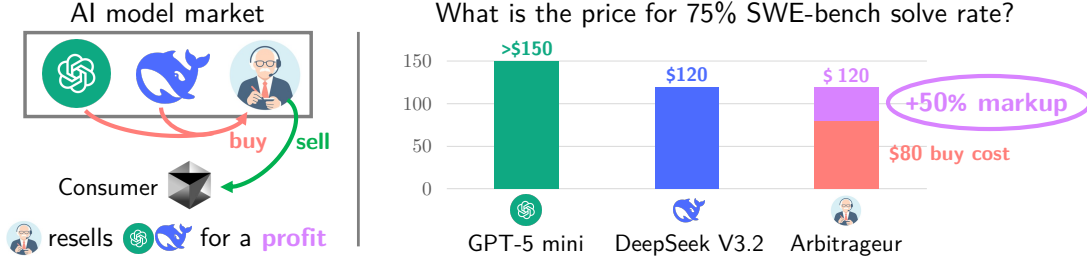


Figure 1. Consider a model market with three providers: GPT-5 mini, DeepSeek v3.2, and an arbitrageur. Consumers demand a target level of performance on SWE-bench-type tasks; for example, a 75% SWE-bench solve rate. Through repeated sampling, GPT-5 mini and DeepSeek each achieve a 75% SWE-bench solve rate at costs of \$150 and \$120, respectively. The arbitrageur instead sources generations by first querying GPT-5 mini (at up to \$0.08 per problem) and, if that fails, querying DeepSeek. Using this strategy, the arbitrageur attains the same 75% solve rate at a cost of \$80. This cost advantage creates a profit opportunity: the arbitrageur can resell its sourced generations at markups of up to 50% while still undercutting the market.

arise when a market participant can simultaneously buy and sell API calls from a combination of multiple providers at a profit, while incurring no model-development risk.

- **We demonstrate the feasibility of computational arbitrage** via an in-depth case study of SWE-bench GitHub issue resolution with GPT-5 mini and DeepSeek v3.2 models. In this setting, simple arbitrage strategies yield net profit margins of up to 40%. We additionally demonstrate that arbitrage is robust: arbitrage policies are inexpensive to fit and remain profitable under distribution shifts.
- **We analyze the economic implications of computational arbitrage.** Competition among arbitrageurs drives down consumer prices, at the expense of providers’ marginal revenues. At the same time, arbitrage reduces market segmentation, even allowing smaller models to capture some of the revenue generated by consumer demand for frontier model performance.
- **Model distillation gives rise to arbitrage opportunities.** Through our own scaling experiments, we show that increased distillation consistently improves cost-to-solution, thereby creating increasingly profitable arbitrage opportunities. We then show that distillation can directly undermine the teacher model’s revenue, potentially eliminating it altogether. To do so, we train mini-coder 4B, a small model that outperforms Qwen Coder 30B in terms of cost-to-solution.

2. Computational arbitrage

We consider an AI model marketplace in which consumers can choose among multiple model providers. Providers take in queries $x \in \mathcal{X}$ (e.g., a software issue description)

together with an inference budget $b \in \mathbb{R}_+$. They then return some output $y \in \mathcal{Y}$ (e.g., a proposed fix) while charging the consumer some cost $c \leq b$. Formally, we model each provider p as a conditional distribution $p(y, c | x, b)$. Consumers derive utility from providers’ outputs and will seek to query the provider that offers the best trade-off between utility and cost.

We evaluate market providers using a standardized performance metric $u : \mathcal{X} \times \mathcal{Y} \rightarrow \mathbb{R}$ (e.g., accuracy), and assume that consumer utility increases monotonically with model performance. For a given query distribution $x \sim D$ and provider p , varying the inference budget b induces different trade-offs between expected cost $\bar{c}_p$ and expected performance $\bar{u}_p$, or *cost* and *performance* for short, specifically

$$\bar{c}_p(b) = \mathbb{E}_{x \sim D, c \sim p(\cdot | x, b)} [c], \quad (1)$$

$$\bar{u}_p(b) = \mathbb{E}_{x \sim D, y \sim p(\cdot | x, b)} [u(x, y)]. \quad (2)$$

The minimum cost required for a provider p to achieve a target performance level u is

$$C_p(u) = \min_{b \text{ s.t. } \bar{u}_p(b) \geq u} \bar{c}_p(b). \quad (3)$$

A fully informed, rational consumer will choose the lowest-cost provider. For a market of providers $\mathbf{P} := \{p_1, p_2, \dots\}$, we define the *market price* $C_{\mathbf{P}}(u)$ for performance level u as the minimum cost at which any provider offers that level of performance:

$$C_{\mathbf{P}}(u) = \min_{p \in \mathbf{P}} C_p(u). \quad (4)$$

As we will see, arbitrageurs seek to obtain below-market prices, thereby creating profit opportunities.

Computational arbitrage. An arbitrageur is a market participant who resells other providers’ outputs for a profit. Specifically, given a query $x \in \mathcal{X}$ and a budget $b \in \mathbb{R}$, an

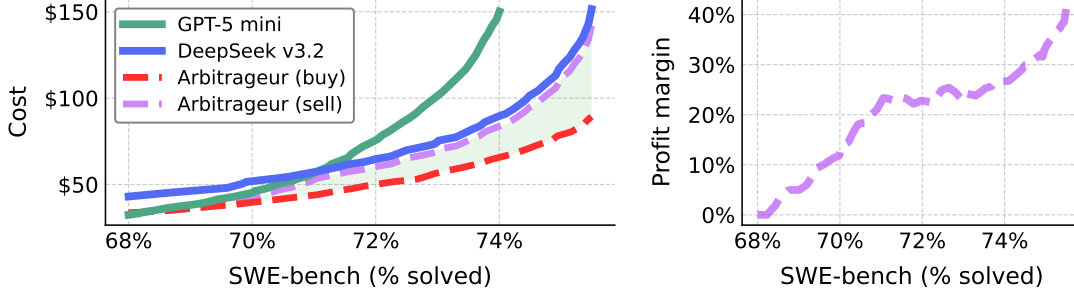


Figure 2. Inference cost for GPT-5 mini and DeepSeek v3.2 to reach different SWE-bench performance levels, with inference budgets scaled through repeated sampling (up to \$1 per issue). We also evaluate the following arbitrage policy: allocate up to \$0.08 to GPT-5 mini and, if it fails, spend the remaining \$0.92 on DeepSeek. The arbitrage policy (red) achieves solve rates above 68% at a lower cost than either GPT-5 mini or DeepSeek. This cost advantage enables the arbitrageur to profit by reselling its generations close to market price.

arbitrageur purchases one or more model responses from the market $\mathbf{P}$, incurring some cost $c \in \mathbb{R}_+$. The arbitrageur then returns one of the acquired responses $y \in \mathcal{Y}$ to the consumer, applying some cost markup $\delta > 0$. We abstract the arbitrageur’s policy for sourcing generations from the market as a conditional distribution $q(y, c \mid x, b)$.

An arbitrageur cannot operate at a loss. At worst, it is unable to offer competitive prices, thus failing to attract demand and earning zero profit. That is, computational arbitrage is risk-free by construction. To profit, however, arbitrageurs must achieve prices lower than those otherwise available in the market.

Definition 2.1 (Arbitrage Opportunity). An *arbitrage opportunity* exists in a marketplace $\mathbf{P}$ under a query distribution D if there exists an arbitrage policy q such that the policy achieves some level of performance at a cost strictly lower than its market price. Formally,

$$\exists \text{ arbitrage policy } q, u \in \mathbb{R} \quad \text{s.t.} \quad C_q(u) < C_{\mathbf{P}}(u), \quad (5)$$

where $C_q(u)$ denotes the arbitrageur’s expected cost of achieving the target performance level u , and $C_{\mathbf{P}}(u)$ denotes the market price for the same performance level.

Intuitively, the arbitrageur can capture the spread between the prevailing market price $C_{\mathbf{P}}(u)$ for the level of performance u and its cost $C_q(u)$ of purchasing that same level of performance. Specifically, for any given performance level u , an arbitrage policy q can earn a marginal profit of

$$\Pi_q(u) = \max(C_{\mathbf{P}}(u) - C_q(u), 0) \geq 0. \quad (6)$$

By construction, arbitrageurs cannot operate at a loss, since at worst they simply fail to attract any demand.

Arbitrageurs seek to maximize profit. The arbitrage policy q^* that maximizes profit in the market is

$$q^* = \arg \max_q \int_{\mathbb{R}} \Pi_q(u) w(u) du, \quad (7)$$

where the weighting function $w(u)$ captures the market demand for any given level of performance u . For simplicity, we will assume that consumer demand is uniform across all performance levels, that is, $w(u) = 1$.

In summary, arbitrageurs achieve below-market prices by optimally sourcing generations from the market, thereby creating opportunities for profit. In the next section, we present an empirical study of computational arbitrage in software issue resolution.

3. Arbitrage in software issue resolution

We focus on SWE-bench Verified (Jimenez et al., 2024; Chowdhury et al., 2024), the leading benchmark for software issue resolution. It comprises 500 software issues sourced from GitHub, each paired with unit tests to verify the functional correctness of model-generated patches. Model performance is measured as the fraction of issues for which the model produces a successful patch. For our initial exposition, we compare GPT-5 mini (Singh et al., 2025) and DeepSeek v3.2 Thinking (Liu et al., 2025). We conduct additional experiments for Lean4 formal theorem proving, see Appendix D.

We scale the inference budget through repeated sampling. Specifically, a model is repeatedly queried to solve a target software issue until it produces a successful patch or exhausts its inference budget (Chen et al., 2021; Li et al., 2022). Further details on the evaluation set-up and model pricing are in Appendices B and C.

For each model i and issue j , we observe n_{ij} solution attempts and m_{ij} correct solutions, along with the mean cost per attempt $\hat{s}_{ij}$. From these, we estimate the probability that the issue is solved within k independent attempts using the standard unbiased estimator $\text{pass}@k = 1 - \binom{n-m}{k} / \binom{n}{k}$. To express performance in terms of monetary cost rather than number of attempts, we convert a dollar budget b into an equivalent number of attempts $k = b / \hat{s}_{ij}$, yielding a

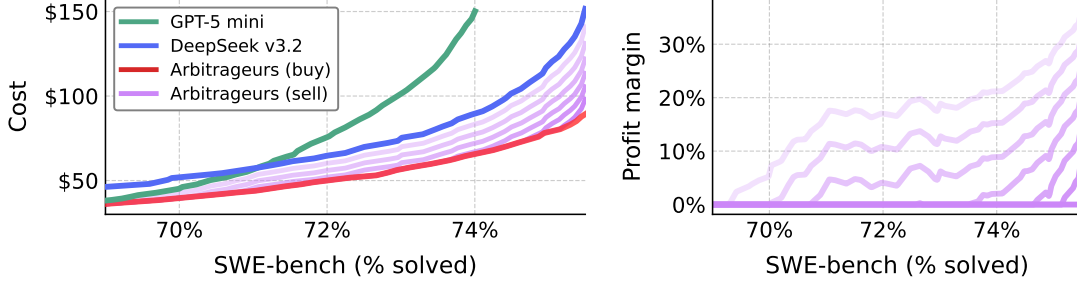


Figure 3. Two arbitrageurs deploy the same arbitrage policy but compete on price. They take turns updating their prices to undercut each other. Earlier turns are plotted with greater transparency. *Left:* Competition between arbitrageurs drives market prices downward. In equilibrium, the market price equals the arbitrageurs’ buy price. *Right:* While arbitrage is initially highly profitable, profit opportunities eventually vanish.

per-issue performance curve $u_{ij}(b) = \text{pass}@ (b/\hat{s}_{ij})$. Aggregating across issues gives the model’s expected solve rate at budget b , that is, $\bar{u}_i(b) = \frac{1}{|J|} \sum_{j \in J} u_{ij}(b)$.

We then compute each model’s expected cost at different budgets b . Specifically, the expected total cost is $c_i(b) = |J| \int_0^b (1 - \bar{u}_i(x)) dx$, which follows from the survival-function identity for non-negative random variables. We plot in Figure 2 (left) inference cost c_i versus performance $\bar{u}_i$ for GPT-5 mini and DeepSeek. GPT-5 mini is more cost-efficient for lower budgets, while DeepSeek is preferable for higher budgets.

Arbitrage policy. We construct arbitrage policies using a model cascade design (Varshney & Baral, 2022), in which market providers are queried sequentially in a fixed order until a successful patch is obtained. The central challenge is determining how to allocate the inference budget optimally across the model cascade.

We denote by $\tau_i \in \mathbb{R}$ the cap on spending for provider p_i . Given a total budget b , provider i is allocated

$$b_i^{(\tau)} = \min\left(\max(b - \sum_{k < i} \tau_k, 0), \tau_i\right), \quad (8)$$

i.e., whatever remains of b after the preceding providers have each claimed up to their cap, clamped to $[0, \tau_i]$.

Each provider independently attempts every unsolved issue using its allocated budget, so the probability that issue j is solved by at least one provider in the cascade is

$$u_j^{(\tau)}(b) = 1 - \prod_{i=1}^{|I|} (1 - u_{i,j}(b_i^{(\tau)})), \quad (9)$$

where $u_{i,j}(b_i)$ is the solve probability of model i on issue j when given budget b_i . Averaging over issues yields the cascade’s expected performance at budget b , that is, $\bar{u}^{(\tau)}(b) = \frac{1}{|J|} \sum_{j \in J} u_j^{(\tau)}(b)$.

As shown earlier, GPT-5 is more cost-effective than DeepSeek for low budgets. Therefore, we build the cascade by first querying GPT-5 mini and then DeepSeek. Specifically, we search for the profit-maximizing allocation τ^* according to Equation 7. Under a maximum inference budget of \$1, this yields $\tau^* = \{\$0.08, \$0.92\}$, meaning that the arbitrageur allocates up to \$0.08 per issue to GPT-5 mini before querying DeepSeek.

We compare the cost-performance of the arbitrageur against GPT-5 mini and DeepSeek in Figure 2 left. The arbitrageur (red curve) can source any given level of performance above 68% solve rate at a cheaper cost than either of the two models. These efficiency gains generate opportunities for profit, as the arbitrageur can undercut the market by pricing its outputs slightly below the market price (pink line). By following this strategy, the arbitrageur can achieve profit margins of up to 40%, as shown in Figure 2 right. The remarkable profitability of arbitrage highlights the inefficiency of querying either GPT-5 mini or DeepSeek alone.

3.1. Economic implications of computational arbitrage

Competition between arbitrageurs reduces prices. We have shown how an arbitrageur can profit by entering an inefficient market. However, even after this entry, arbitrage opportunities may still remain. In particular, a competing arbitrageur could enter the market and undercut the first by accepting a smaller profit margin.

Consider two arbitrageurs who source their outputs from the profit-maximizing arbitrage policy q^* , and compete over pricing. The two arbitrageurs take turns updating their prices, so as to be cheaper than the prevailing market price. We plot in Figure 3 left the market’s cost-performance frontier as the two arbitrageurs sequentially update their prices. By undercutting each other, market prices reduce considerably. As a result, arbitrage profitability decreases, as plotted in Figure 3, and arbitrage ultimately ceases to be profitable.

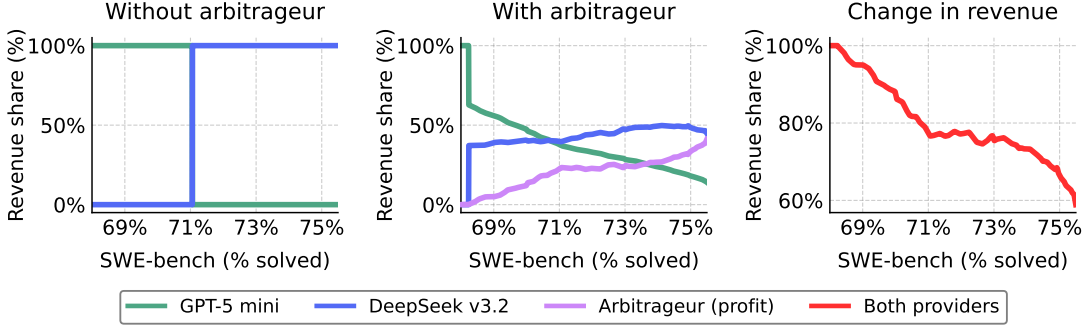


Figure 4. Revenue split across model providers for different levels of performance. *Left*: In the absence of arbitrageurs, the market is segmented by performance, with a single model dominating each segment. *Middle*: Arbitrageurs eliminate this segmentation, allowing both models to earn revenue across a much broader range of levels of performance. *Right*: Arbitrageurs reduce providers’ marginal revenue, with the lost surplus transferred to arbitrageur profits or passed on to consumers as lower market prices.

These dynamics correspond to the classic framework of Bertrand competition (Mas-Colell et al., 1995). When two providers have identical marginal costs, offer identical products, and consumers have perfect information of providers’ prices, the equilibrium outcome is one in which market price equals marginal cost. By construction, the two competing arbitrageurs share identical marginal costs C_{q^*} . Consequently, the equilibrium market price of the new market $\mathbf{P}'$ is at most the arbitrageurs’ marginal cost, that is,

$$C_{\mathbf{P}'}(u) = \min(C_{\mathbf{P}}(u), C_{q^*}(u)). \quad (10)$$

As a result, the profit-maximizing arbitrage policy q^* for the original market $\mathbf{P}$ ceases to be profitable in the new market $\mathbf{P}'$. In this sense, arbitrage is self-defeating. When arbitrage opportunities exist, competition among arbitrageurs quickly eliminates them. From the consumer side, the consequence is lower market prices. We next examine the economic implications of arbitrage for model providers.

Arbitrage breaks market segmentation and reduces providers’ revenue In this section, we study the implications of computational arbitrage for market segmentation. As discussed previously, GPT-5 mini is more cost-effective for lower inference budgets, whereas DeepSeek is more cost-effective for higher budgets. This implies that the market is segmented into two distinct performance tiers, as shown in Figure 4 left. Consumers seeking performance below a 71% SWE-bench solve rate should query GPT-5 mini, while those seeking higher performance should query DeepSeek.

When the arbitrageur enters the market, it captures all consumer demand by offering lower prices. Nevertheless, both providers may continue to earn revenue, since the arbitrageur relies on them for sourcing its outputs. We plot in Figure 4 middle how consumer expenditure is split across providers once the arbitrageur enters the market. Notably, market segmentation disappears. Instead, DeepSeek earns

revenue across a wider range of SWE-bench performance levels. Similarly, GPT-5 mini earns revenue across the entire performance spectrum, including at the frontier (i.e., a 75% SWE-bench solve rate).

This latter observation has important implications. In markets where consumers only seek frontier performance, cheaper models such as GPT-5 mini are not irrelevant. On the contrary, by contributing to overall efficiency, cheap models can earn revenue even at the performance frontier. Therefore, successful market entry does not require offering the best-performing model; being sufficiently cheap can suffice.

Arbitrage profits come at the expense of providers’ marginal revenue. In our setting, marginal revenue decreases by up to 40%, as plotted in Figure 4 right. This revenue loss is either transformed into arbitrage profits or, in the presence of competing arbitrageurs, passed on to consumers as lower prices¹.

Arbitrage is inexpensive and robust. As a theoretical concept in economics, arbitrage should require no initial investment and entail no risk (Hull, 1989). In practice, however, arbitrageurs necessarily incur costs (Shleifer & Vishny, 1995). We now show that computational arbitrage is practical: profitable policies are inexpensive to find and generalize well.

Identifying arbitrage opportunities requires collecting a dataset of cost comparisons across a number of input queries. This dataset is then used to fit an arbitrage policy that maximizes expected profit. The cost comparisons can be collected while serving GPT-5 mini, with the only additional search cost arising from redundant queries to DeepSeek. We allow a search budget of \$0.5 per query. Consequently, in

¹Note that an overall reduction in prices may result in higher trading volumes, and thus larger total revenue.

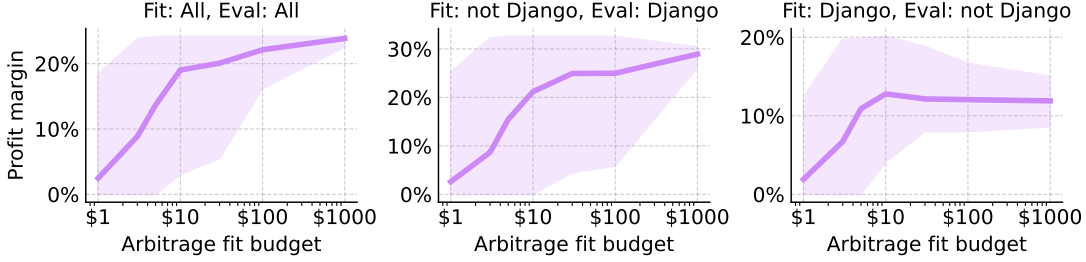


Figure 5. Profit margin across different search budgets when fitting the arbitrage policy. The solid line represents mean profitability, whereas the shaded area indicates the 95% confidence interval, computed by bootstrapping over the samples acquired within the search budget. *Left:* When fitting a fixed query distribution, small search budgets (e.g., \$10) consistently yield profitable arbitrage policies. *Middle and right:* We fit the arbitrageur either on software issues from the Django library or on issues from other repositories, and evaluate the resulting arbitrage policy on the held-out data. We find that, on expectation, the arbitrageur remains profitable.

the worst case, a total search budget of \$10 permits only $10 / \$0.50 = 20$ price comparisons.

We plot in Figure 5 (left) the profitability of the fitted policy as a function of the search budget. We report mean profitability over the 70% to 75% SWE-bench performance range. The solid line represents mean profitability, whereas the shaded area indicates the 95% confidence interval, computed by bootstrapping over the samples acquired within the search budget. In expectation, budgets as low as \$1 suffice to yield profitable arbitrage policies. However, the 95% confidence intervals are wide due to the small sample sizes. A slightly larger search budget of \$10 allows for consistently fitting a profitable policy. Therefore, the initial investment required for computational arbitrage is minimal.

We then split SWE-bench into issues from the Django repository and issues from all other repositories. We select Django because it accounts for roughly half of all SWE-bench issues. This split induces a natural distribution shift in the query distribution: Django primarily concerns web development, which differs substantially from other SWE-bench domains (e.g., scientific computing with scikit-learn). It also introduces a difficulty shift, as Django issues tend to be easier.

We fit arbitrage policies on one of the SWE-bench splits, and evaluate their profitability on the other split. We plot in Figure 5 (middle and right) out-of-distribution profitability against in-distribution search cost. We report mean profitability on the upper end of model performance, that is, 75%-80% solve rate for Django issues, and 61%-66% solve rate for non-Django issues. On expectation, arbitrage policies remain profitable even with search expenditures as low as \$1. At larger search budgets (e.g., \$30), the learned policies are consistently profitable. That is, the profit-maximizing arbitrage policy for each query distribution generalizes and remains profitable under reasonably large distribution shifts.

Arbitrage in larger model markets. So far, we have demonstrated arbitrage opportunities in a two-provider market. We now examine how these opportunities change when four additional model providers of varying sizes enter the market: Qwen 3 Coder 30B and 480B (Hui et al., 2024; Team, 2025), Claude Sonnet 4.5 (Anthropic, 2025), and our distilled mini-coder 4B model, trained using the distillation procedure described in Section 4.

We plot the cost–performance curves of the six models in Figure 6 (left). Compared to GPT-5 mini and DeepSeek, mini-coder and Qwen Coder 30B are considerably smaller and therefore more efficient at low inference budgets. In contrast, Claude Sonnet 4.5 is too expensive in the compute regimes we consider; a \$1 budget is often insufficient to submit a solution for many SWE-bench problems. Finally, Qwen Coder 480B is dominated by GPT-5 mini and DeepSeek, as it is neither particularly cost-efficient nor high-performing.

We next examine the revenue earned by each model in the market. For each performance level between 45% and 75% SWE-bench solve rate, we search for the arbitrage policy with the lowest purchase cost. This cost represents the arbitrage-free (i.e., equilibrium) market price for that level of performance. We plot the corresponding revenue shares in Figure 6 (middle). The arbitrage-free market is not segmented: four of the six providers share revenue along the performance frontier (e.g., above a 70% SWE-bench solve rate). Two models are uncompetitive and earn no revenue: Claude Sonnet 4.5 is too expensive for the budget regimes considered, while Qwen Coder 480B is neither sufficiently cost-efficient nor high-performing.

Lastly, we compare the profitability of arbitrage in the six-model market with that in the previously analyzed two-model market (see Figure 6, right). In the six-model market, arbitrage opportunities emerge at a 42% SWE-bench solve rate, compared to 68% in the two-model market. Moreover, arbitrage in the six-model market is strictly more profitable,

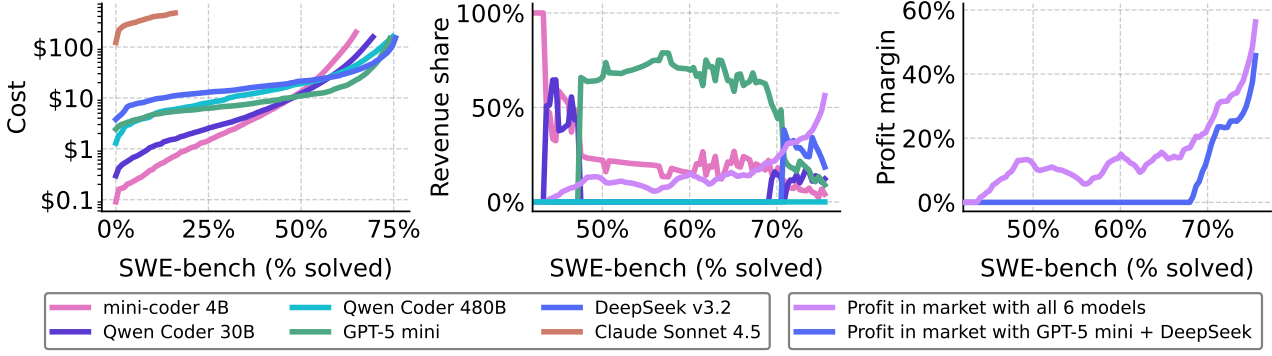


Figure 6. Model market with six providers of varying sizes. *Left:* Cost for each model to achieve different levels of SWE-bench performance. *Middle:* Revenue split across providers. There is little market segmentation, with up to four providers sharing revenue at a given performance level. Some models, such as Qwen Coder 480B and Claude Sonnet 4.5, are not competitive. *Right:* Compared with the earlier two-model market (GPT-5 mini and DeepSeek), the six-model market yields more, and more profitable, arbitrage opportunities.

with margins reaching up to 58%, versus 45% in the two-model setting. Thus, arbitrage opportunities are both more prevalent and more profitable in the six-model market.

In summary, larger markets are not necessarily more efficient. On the contrary, access to a broader set of providers with varying cost-efficiency can favor arbitrageurs, resulting in arbitrage opportunities across a wider range of performance levels and increased profitability. In the next section, we examine the effectiveness of distillation for training models with different cost-efficiency trade-offs, thereby enabling arbitrage.

4. Distillation and arbitrage.

Arbitrageurs exploit cost differentials to generate profit opportunities. Model distillation compresses the capabilities of a large teacher into a smaller, cheaper student. In this section, we examine how distillation facilitates arbitrage. First, we show that arbitrage profitability grows monotonically with the distillation budget. Second, we show that distilled models can substantially erode the teacher model’s revenue.

4.1. Distillation creates arbitrage opportunities

We use Qwen Coder to distill small 1.7B models at different distillation budgets. We then analyze the profitability of arbitrage when pairing each distilled model with Qwen Coder 480B.

To synthesize the training data, we start from SWE-Smith (Yang et al., 2025), a dataset of over 60k GitHub issues. Approximately 80% of the issues lack descriptions, which we generate using Qwen 3 235B Instruct. We discard 13% of issues due to Docker compatibility problems, yielding a final set of 52.4k distinct GitHub issues. We

then use Qwen Coder 30B² to generate eight trajectories per issue, yielding about 400k training trajectories or 5.4B training tokens. Although only 20% of these trajectories are correct, training on the full set of trajectories results in better downstream performance.

We distill five Qwen 3 1.7B models at different data scales: 70M, 200M, 600M, 1.8B, and 5.4B training tokens, using standard supervised fine-tuning (SFT). We then evaluate their pass@ k performance on SWE-bench, see Figure 7 (left). We find that increased distillation consistently improves pass@ k . In fact, distillation leads to larger improvements in pass@100 compared to pass@1. In turn, models distilled on more data dominate those distilled on less data in terms of their cost-performance, see Figure 7 (middle).

Next, we evaluate the arbitrage opportunities enabled by each distilled model when paired with Qwen Coder 480B. We measure mean profitability on the upper end of performance (61% to 71% SWE-bench), and plot mean profitability against the number of distillation tokens in Figure 7 right. We observe that increased distillations consistently create more profitable arbitrage opportunities, with profitability increasing roughly log-linearly with the number of distillation tokens. The model distilled on 5.4B tokens (400k examples) enables a remarkably high level of profitability, allowing for a profit margin of nearly 30%.

Therefore, distillation is highly effective at creating arbitrage opportunities. We replicate these scaling experiments in the setting of Lean 4 formal theorem proving and observe consistent findings, see Appendix D.1. Next, we examine how the teacher model’s revenue changes after distilled models enter the market.

²We use the 30B model to reduce the cost of data generation. We would expect better performance when using the 480B model.

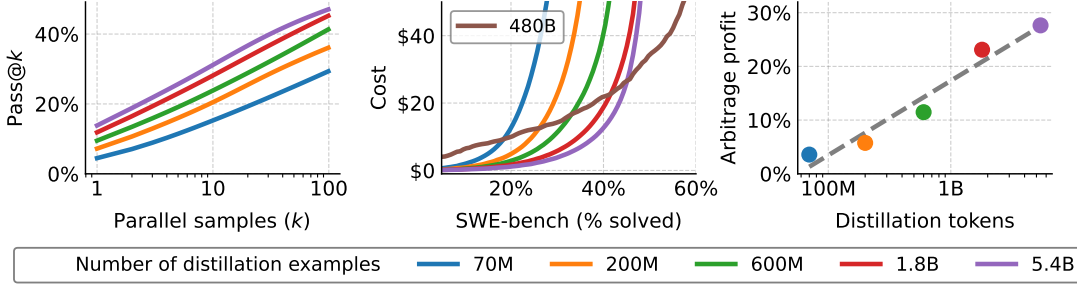


Figure 7. We fine-tune Qwen 3 1.7B using data generated by Qwen Coder 30B. We distill for up to 400k examples (5.4B tokens). *Left*: Models distilled on more data Pareto-dominate in terms of pass@ k . *Middle*: Models distilled on more data attain higher levels of performance at lower cost. *Right*: When paired with Qwen Coder 480B, models distilled on more data create increasingly more profitable arbitrage opportunities.

4.2. Distillation and revenue displacement

Having established the effectiveness of distillation but lacking additional seed problems to generate more training data, we turn to scaling model size. Specifically, we fine-tune Qwen 3 4B on the full 400k training examples from the previous section and refer to the resulting model as mini-coder 4B. We then examine how the introduction of mini-coder 4B affects the revenue of Qwen Coder 30B under competition.

To do so, we analyze a three-way market consisting of mini-coder 4B, Qwen Coder 30B, and the more capable GPT-5 mini model. We compare the pass@\$ k performance of the three models in Figure 8 left, where pass@\$ k denotes the solve rate under a \$ k budget per example. As expected, both mini-coder and Qwen Coder are more cost-efficient than GPT-5 mini at small sampling budgets. However, in this low-budget regime, mini-coder outperforms Qwen Coder; consequently, Qwen Coder is dominated by mini-coder and GPT-5 mini.

To assess whether mini-coder could serve as a replacement for Qwen Coder, we consider two market configurations: GPT-5 mini paired with Qwen Coder, and GPT-5 mini paired with mini-coder. We plot the arbitrage-free market price (i.e., the arbitrage buy cost) in each setting in Figure 8 middle. The market with mini-coder yields prices comparable to those in the Qwen Coder market, indicating that the distilled model could effectively replace its teacher model without increasing market prices.

We now examine how the entry of mini-coder affects Qwen Coder’s revenue. In Figure 8 (left), we plot Qwen Coder’s revenue share in two settings: before mini-coder enters the market (i.e., GPT-5 mini + Qwen Coder 30B) and after mini-coder enters (i.e., the three-model market). Before mini-coder’s entry (brown), Qwen Coder captures nearly all revenue in the lower-performance regime and maintains a substantial share of around 20% at the performance frontier. After mini-coder enters (blue), Qwen Coder’s revenue nearly disappears, with the model earning revenue only within

a narrow performance band. In other words, mini-coder cannibalizes nearly all of its teacher model’s revenue.

These results highlight the effectiveness of distillation. Distilled models can outperform their teacher models precisely in the performance regimes that matter most given existing competitors. In doing so, they may cannibalize a large share of the teacher model’s revenue.

5. Related work

Pricing model outputs. Price-per-token pricing is ubiquitous in current model marketplaces. Prior work has identified several potential pitfalls, including token count misrepresentation (Velasco et al., 2025a;b; Wang et al., 2025b; Sun et al., 2025) and model substitution (Gao et al., 2025; Cai et al., 2025; Sun et al., 2025). Alternative pricing mechanisms have also been proposed, such as pay-for-performance contracts (Saig et al., 2024), second-best performance auctions (Cao et al., 2025), and menus of two-part tariffs (Bergemann et al., 2025). Our model is agnostic to the specific mechanism used by each provider to price model outputs, provided that cost–performance curves can be computed. Instead, we study the extent to which arbitrageurs can exploit price differentials in the market, which in turn enables us to determine overall market efficiency and the arbitrage-free valuation of different performance levels.

Model cascading and routing. Model cascading sequentially queries multiple models, typically in increasing order of cost, until a response satisfying a predefined quality criterion is obtained (Varshney & Baral, 2022; Wang et al., 2023; Madaan et al., 2023; Chen et al., 2023; Ramírez et al., 2024; Zhang et al., 2024; Kapoor et al., 2025). Model routing, by contrast, assigns different queries to different models in order to maximize performance, minimize cost, or a combination of both (Shnitzer et al., 2024; Ding et al., 2024; Lu et al., 2024; Šakota et al., 2024). Arbitrageurs may draw upon the literature on model cascading and model routing to

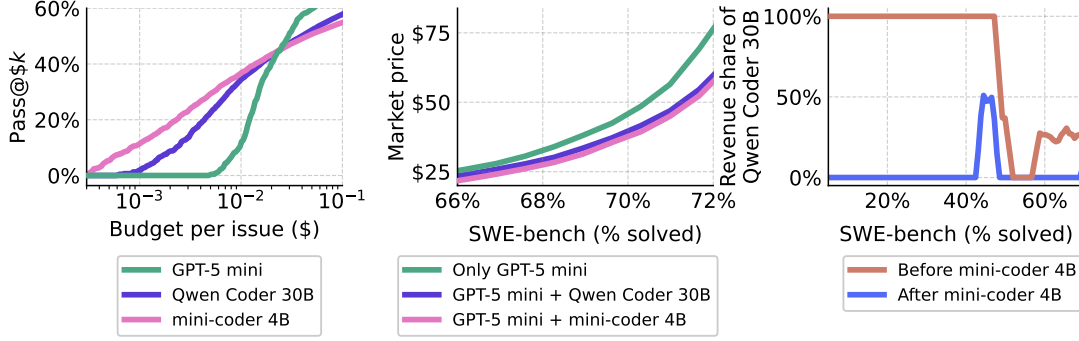


Figure 8. We train mini-coder 4B with data generated by Qwen Coder 30B. *Left*: mini-coder outperforms Qwen Coder at inference budgets up to \$0.02 per issue. *Middle*: When paired with GPT-5 mini, mini-coder leads to lower market prices (i.e., arbitrage buy costs) than Qwen Coder. *Right*: Upon entering a market consisting of GPT-5 mini and Qwen Coder, mini-coder cannibalizes nearly all of Qwen Coder’s revenue.

construct arbitrage policies. Conversely, research on model routing and cascading can adopt arbitrage profitability as key benchmark for algorithm development.

Distillation. We examine how test-time scaling capabilities transfer through distillation. While modest amounts of distillation can improve pass@ k (Chen et al., 2021; Yue et al., 2025), excessive fine-tuning may lead to diversity collapse and reduced pass@ k performance (Cobbe et al., 2021; Chen et al., 2025; Dang et al., 2025). To mitigate this issue, inference-aware fine-tuning methods have been proposed (Chow et al., 2025; Chen et al., 2025; Goyal et al., 2025; Dang et al., 2025). In contrast, we distill models using standard supervised fine-tuning with the cross-entropy loss. We find that test-time scaling performance (e.g., pass@100) continues to improve as the number of distillation tokens scales into the billions.

More broadly, distillation typically produces models that are less capable but cheaper to run. This makes it difficult to assess their value, given that a more powerful teacher model is necessarily also available. Our arbitrage framework allows us to quantify the economic value of distilled models by measuring their revenue upon market entry.

Economic analyses of AI model ecosystems. Prior work examines various economic aspects of model ecosystems. Erol et al. (2025) analyze differences in the expected cost of producing a correct output across language models. In contrast, we examine how arbitrageurs can exploit these cost differentials and study the resulting economic implications. Bergemann et al. (2025) develop an economic framework for optimal pricing of model inference and fine-tuning; we instead focus on the arbitrage-free valuation of model performance. Xu et al. (2025) investigate how different economic factors shape model providers’ openness decisions; arbitrage may incentivize dominant providers to gatekeep small,

efficient models. Finally, Jagadeesan et al. (2025) show that multi-objective model development lowers barriers to market entry. We focus on cost-efficiency rather than safety, and demonstrate how arbitrage facilitates market entry.

6. Discussion

We initiate the study of computational arbitrage in AI model markets. We demonstrate its feasibility through a case study on SWE-bench GitHub issue resolution, and analyze several economic implications, such as reductions in market prices and lowered barriers to entry. We then study the interplay between computational arbitrage and model distillation: the existence of arbitrageurs incentivizes the market entry of distilled models, which in turn create the very opportunities these arbitrageurs exploit.

Many theoretical and empirical questions remain open. In this work we assume verification to be costless. An important research avenue is to investigate the implications of costly or imperfect verification (Patwardhan et al., 2025). We have also assumed perfect market information. The cost of market information (Stigler, 1962) may determine whether computational arbitrage meaningfully improves market efficiency or merely shifts market concentration from model providers toward oligopolistic intermediaries.

From a technical perspective, the arbitrage strategies we consider are both query-agnostic and user-agnostic. While already remarkably effective, practitioners may draw on insights from the rich literature on model routing to devise more sophisticated arbitrage policies. Active learning approaches could allow arbitrageurs to dynamically adapt to evolving query distributions and market conditions. These technical improvements, insofar as they promote market efficiency, stand to benefit consumers and providers alike; the former through lower prices, the latter through lowered barriers to entry.

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none which we feel must be specifically highlighted here.

References

- Anthropic. Claude sonnet 4.5 system card, 2025. URL <https://www-cdn.anthropic.com/963373e433e489a87a10c823c52a0a013e9172dd.pdf>.
- Bergemann, D., Bonatti, A., and Smolin, A. The economics of large language models: Token allocation, fine-tuning, and optimal pricing. In *Proceedings of the 26th ACM Conference on Economics and Computation*, pp. 786–786, 2025.
- Brown, B., Juravsky, J., Ehrlich, R. S., Clark, R., Le, Q. V., Re, C., and Mirhoseini, A. Large language mon-keys: Scaling inference compute with repeated sampling, 2025. URL <https://openreview.net/forum?id=0xUEBQV54B>.
- Cai, W., Shi, T., Zhao, X., and Song, D. Are you getting what you pay for? auditing model substitution in llm apis. *arXiv preprint arXiv:2504.04715*, 2025.
- Cao, Y., Wang, Y., Liu, S., Li, M., Tao, Y., and He, T. Pay for the second-best service: A game-theoretic approach against dishonest llm providers. *arXiv preprint arXiv:2511.00847*, 2025.
- Chen, F., Raventos, A., Cheng, N., Ganguli, S., and Druckmann, S. Rethinking fine-tuning when scaling test-time compute: Limiting confidence improves mathematical reasoning. *arXiv preprint arXiv:2502.07154*, 2025.
- Chen, L., Zaharia, M., and Zou, J. Frugalgpt: How to use large language models while reducing cost and improving performance. *Transactions on Machine Learning Research*, 2023.
- Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P. D. O., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- Chernova, Y. Openrouter, a marketplace for ai models, raises \$40 million. *The Wall Street Journal*, June 2025.
- Chow, Y., Tennenholtz, G., Gur, I., Zhuang, V., Dai, B., Kumar, A., Agarwal, R., Thiagarajan, S., Boutilier, C., and Faust, A. Inference-aware fine-tuning for best-of-n sampling in large language models. In *The Thirteenth International Conference on Learning Representations*, 2025.
- Chowdhury, N., Aung, J., Shern, C. J., Jaffe, O., Sherburn, D., Starace, G., Mays, E., Dias, R., Aljube, M., Glaese, M., Jimenez, C. E., Yang, J., Ho, L., Patwardhan, T., Liu, K., and Madry, A. Introducing SWE-bench verified, 2024. URL <https://openai.com/index/introducing-swe-bench-verified/>.
- Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- Dang, X., Baek, C., Wen, K., Kolter, Z., and Raghunathan, A. Weight ensembling improves reasoning in language models. *arXiv preprint arXiv:2504.10478*, 2025.
- Ding, D., Mallick, A., Wang, C., Sim, R., Mukherjee, S., Rühle, V., Lakshmanan, L. V., and Awadallah, A. H. Hybrid llm: Cost-efficient and quality-aware query routing. In *The Twelfth International Conference on Learning Representations*, 2024.
- Erol, M. H., El, B., Suzgun, M., Yuksekgonul, M., and Zou, J. Cost-of-pass: An economic framework for evaluating language models. *arXiv preprint arXiv:2504.13359*, 2025.
- Gao, I., Liang, P., and Guestrin, C. Model equality testing: Which model is this api serving? *ICLR*, 2025.
- Goyal, S., Lopez-Paz, D., and Ahuja, K. Distilled pre-training: A modern lens of data, in-context learning and test-time scaling. *arXiv preprint arXiv:2509.01649*, 2025.
- Guo, D., Yang, D., Zhang, H., Song, J., Zhang, R., Xu, R., Zhu, Q., Ma, S., Wang, P., Bi, X., et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- Hardt, M. and Sun, Y. Test-time training on nearest neighbors for large language models. In *The Twelfth International Conference on Learning Representations*, 2024.
- Hassid, M., Remez, T., Gehring, J., Schwartz, R., and Adi, Y. The larger the better? improved llm code-generation via budget reallocation. In *First Conference on Language Modeling*, 2024.
- Hoffmann, J., Borgeaud, S., Mensch, A., Buchatskaya, E., Cai, T., Rutherford, E., de Las Casas, D., Hendricks, L. A., Welbl, J., Clark, A., et al. Training compute-optimal large language models. In *Proceedings of the 36th International Conference on Neural Information Processing Systems*, pp. 30016–30030, 2022.
- Hui, B., Yang, J., Cui, Z., Yang, J., Liu, D., Zhang, L., Liu, T., Zhang, J., Yu, B., Dang, K., et al. Qwen2. 5-coder technical report. *arXiv preprint arXiv:2409.12186*, 2024.

- Hull, J. *Options, futures, and other derivative securities*. Prentice Hall, United states of America, first edition edition, 1989.
- Jagadeesan, M., Jordan, M. I., and Steinhardt, J. Safety versus performance: How multi-objective learning reduces barriers to market entry. *Proceedings of the National Academy of Sciences*, 122(42):e2510004122, 2025.
- Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., and Narasimhan, K. R. SWE-bench: Can language models resolve real-world github issues? In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=VTF8yNQm66>.
- Kapoor, S., Stroebl, B., Siegel, Z. S., Nadgir, N., and Narayanan, A. AI agents that matter. *Transactions on Machine Learning Research*, 2025. ISSN 2835-8856. URL <https://openreview.net/forum?id=Zy4uFzMviZ>.
- Keles, A. Verifiability is the limit. <https://alperenkeles.com/posts/verifiability-is-the-limit/>, 2025. Accessed: 2026-03-01.
- LI, J., Beeching, E., Tunstall, L., Lipkin, B., Solteskyi, R., Huang, S. C., Rasul, K., Yu, L., Jiang, A., Shen, Z., Qin, Z., Dong, B., Zhou, L., Fleureau, Y., Lample, G., and Polu, S. Numina-math. [<https://huggingface.co/AI-MO/NuminaMath-1.5>] (https://github.com/project-numina/aimo-progress-prize/blob/main/report/numina_dataset.pdf), 2024.
- Li, Y., Choi, D., Chung, J., Kushman, N., Schrittwieser, J., Leblond, R., Eccles, T., Keeling, J., Gimeno, F., Dal Lago, A., et al. Competition-level code generation with alpha-code. *Science*, 378(6624):1092–1097, 2022.
- Liu, A., Mei, A., Lin, B., Xue, B., Wang, B., Xu, B., Wu, B., Zhang, B., Lin, C., Dong, C., et al. Deepseek-v3. 2: Pushing the frontier of open large language models. *arXiv preprint arXiv:2512.02556*, 2025.
- Lu, K., Yuan, H., Lin, R., Lin, J., Yuan, Z., Zhou, C., and Zhou, J. Routing to the expert: Efficient reward-guided ensemble of large language models. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pp. 1964–1974, 2024.
- Madaan, A., Aggarwal, P., Anand, A., Potharaju, S. P., Mishra, S., Zhou, P., Gupta, A., Rajagopal, D., Kappaganthu, K., Yang, Y., et al. Automix: Automatically mixing language models. *CoRR*, 2023.
- Mas-Colell, A., Whinston, M. D., Green, J. R., et al. *Microeconomic theory*, volume 1. Oxford university press New York, 1995.
- Moura, L. d. and Ullrich, S. The lean 4 theorem prover and programming language. In *International Conference on Automated Deduction*, pp. 625–635. Springer, 2021.
- Patwardhan, T., Dias, R., Proehl, E., Kim, G., Wang, M., Watkins, O., Fishman, S. P., Aljube, M., Thacker, P., Fauconnet, L., Kim, N. S., Chao, P., Miserendino, S., Chabot, G., Li, D., Sharman, M., Barr, A., Glaese, A., and Tworek, J. Gdpval: Evaluating ai model performance on real-world economically valuable tasks. *ArXiv*, abs/2510.04374, 2025.
- Ramírez, G., Birch, A., and Titov, I. Optimising calls to large language models with uncertainty-based two-tier selection. In *First Conference on Language Modeling*, 2024.
- Saig, E., Einav, O., and Talgam-Cohen, I. Incentivizing quality text generation via statistical contracts. *Advances in Neural Information Processing Systems*, 37:51196–51222, 2024.
- Šakota, M., Peyrard, M., and West, R. Fly-swat or cannon? cost-effective language model choice via meta-modeling. In *Proceedings of the 17th ACM International Conference on Web Search and Data Mining*, pp. 606–615, 2024.
- Shleifer, A. and Vishny, R. W. The limits of arbitrage. NBER Working Papers 5167, National Bureau of Economic Research, Inc, Jul 1995.
- Shnitzer, T., Ou, A., Silva, M., Soule, K., Sun, Y., Solomon, J., Thompson, N., and Yurochkin, M. Large language model routing with benchmark datasets. In *First Conference on Language Modeling*, 2024.
- Singh, A., Fry, A., Perelman, A., Tart, A., Ganesh, A., El-Kishky, A., McLaughlin, A., Low, A., Ostrow, A., Ananthram, A., et al. Openai gpt-5 system card. *arXiv preprint arXiv:2601.03267*, 2025.
- Stigler, G. J. Information in the labor market. *Journal of political economy*, 70(5, Part 2):94–105, 1962.
- Sun, G., Wang, Z., Zhao, X., Tian, B., Shen, Z., He, Y., Xing, J., and Li, A. Invisible tokens, visible bills: The urgent need to audit hidden operations in opaque llm services. *arXiv preprint arXiv:2505.18471*, 2025.
- Team, Q. Qwen3 technical report, 2025. URL <https://arxiv.org/abs/2505.09388>.
- Varshney, N. and Baral, C. Model cascading: Towards jointly improving efficiency and accuracy of nlp systems.

- In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pp. 11007–11021, 2022.
- Velasco, A. A., Tsirtsis, S., and Gomez-Rodriguez, M. Auditing pay-per-token in large language models. *arXiv preprint arXiv:2510.05181*, 2025a.
- Velasco, A. A., Tsirtsis, S., Okati, N., and Gomez-Rodriguez, M. Is your llm overcharging you? tokenization, transparency, and incentives. *arXiv preprint arXiv:2505.21627*, 2025b.
- Wang, H., Unsal, M., Lin, X., Baksys, M., Liu, J., Dos Santos, M., Sung, F., Vinyes, M., Ying, Z., Zhu, Z., et al. Kimina-prover preview: Towards large formal reasoning models with reinforcement learning. *CoRR*, 2025a.
- Wang, Y., Chen, K., Tan, H., and Guo, K. Tabi: An efficient multi-level inference system for large language models. In *Proceedings of the Eighteenth European Conference on Computer Systems*, pp. 233–248, 2023.
- Wang, Z., Sun, G., He, Y., Shen, Z., Tian, B., and Li, A. Predictive auditing of hidden tokens in llm apis via reasoning length estimation. *arXiv preprint arXiv:2508.00912*, 2025b.
- Wei, J. Asymmetry of verification and verifier’s rule. <https://www.jasonwei.net/blog/asymmetry-of-verification-and-verifiers-law>, 2025.
- Wu, Y., Sun, Z., Li, S., Welleck, S., and Yang, Y. Inference scaling laws: An empirical analysis of compute-optimal inference for llm problem-solving. In *The Thirteenth International Conference on Learning Representations*, 2025.
- Xu, F., Wang, X., Chen, W., and Xie, K. The economics of ai foundation models: Openness, competition, and governance. *arXiv preprint arXiv:2510.15200*, 2025.
- Yang, J., Jimenez, C. E., Wettig, A., Lieret, K., Yao, S., Narasimhan, K. R., and Press, O. SWE-agent: Agent-computer interfaces enable automated software engineering. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. URL <https://arxiv.org/abs/2405.15793>.
- Yang, J., Lieret, K., Jimenez, C. E., Wettig, A., Khandpur, K., Zhang, Y., Hui, B., Press, O., Schmidt, L., and Yang, D. Swe-smith: Scaling data for software engineering agents, 2025. URL <https://arxiv.org/abs/2504.21798>.
- Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T., Cao, Y., and Narasimhan, K. Tree of thoughts: Deliberate problem solving with large language models. *Advances in neural information processing systems*, 36:11809–11822, 2023.
- Yue, Y., Chen, Z., Lu, R., Zhao, A., Wang, Z., Song, S., and Huang, G. Does reinforcement learning really incentivize reasoning capacity in llms beyond the base model? In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025.
- Zhang, J., Krishna, R., Awadallah, A. H., and Wang, C. Ecoassistant: Using llm assistants more affordably and accurately. In *ICLR 2024 Workshop on Large Language Model (LLM) Agents*, 2024.
- Zheng, K., Han, J. M., and Polu, S. minif2f: a cross-system benchmark for formal olympiad-level mathematics. In *International Conference on Learning Representations*, 2022.

A. Extended related work

Inference-time scaling. Inference-time scaling allows model performance to be traded off against inference cost. We scale inference-time compute through repeated sampling (Cobbe et al., 2021; Chen et al., 2021; Li et al., 2022). Other approaches could also be considered, such as test-time search (Yao et al., 2023) or test-time training (Hardt & Sun, 2024). Meaningful comparisons across models and inference-time strategies require a standardized measure of inference cost. The literature typically uses floating-point operations (FLOPs) for this purpose (Hassid et al., 2024; Wu et al., 2025; Brown et al., 2025). Instead, we measure inference cost in USD using OpenRouter’s prices, which more directly reflect usage costs. More importantly, our arbitrage framework offers a novel way to benchmark different combinations of models and inference-time scaling strategies by evaluating how much revenue they can generate in a competitive market.

Verification. For some tasks, verifying the correctness of a solution is easier than generating correct solutions (Cobbe et al., 2021). The more verifiable a task is, the easier it tends to be to improve model performance (Wei, 2025; Koles, 2025). For example, improvements can be achieved at training time through RL with verifiable rewards (Guo et al., 2025), or at test time by scaling inference compute (Chen et al., 2021).

Our work highlights an additional consequence of verifiability: the emergence of AI model markets. When model outputs are verifiable, customers can choose freely among different providers without requiring prior familiarity or trust. Verifiable solutions become fungible goods and are therefore subject to classical economic analysis. As a result, verification facilitates not only higher-performing models but also enables the development of competitive model markets. Although verification may entail costs (Patwardhan et al., 2025), these costs can be incorporated straightforwardly into our economic analysis.

Distillation. We examine how test-time scaling capabilities transfer through distillation. While modest amounts of distillation can improve $\text{pass}@k$ (Chen et al., 2021; Yue et al., 2025), excessive fine-tuning may lead to diversity collapse and reduced $\text{pass}@k$ performance (Cobbe et al., 2021; Chen et al., 2025; Dang et al., 2025). To mitigate this issue, inference-aware fine-tuning methods have been proposed (Chow et al., 2025; Chen et al., 2025; Goyal et al., 2025; Dang et al., 2025). In contrast, we distill models using standard supervised fine-tuning with the cross-entropy loss. We find that test-time scaling performance (e.g., $\text{pass}@100$) continues to improve as the number of distillation tokens scales into the billions.

More broadly, distillation typically produces models that are less capable but cheaper to run. This makes it difficult to assess their value, given that a more powerful teacher model is necessarily also available. Our arbitrage framework allows us to quantify the economic value of distilled models by measuring the revenue they can generate upon market entry, as well as the extent to which they drive down market prices.

B. Evaluation details

We evaluate on a subset of 445 problems from SWE-bench Verified, rather than the full set of 500 issues (i.e., 89% of the benchmark). This restriction is due to incompatibility issues between our local cluster and several of the SWE-bench Docker images, which prevent certain instances from running successfully. These incompatibilities primarily affect issues originating from the matplotlib repository.

For generation, we use the lightweight mini-coder-v1 scaffolding (Yang et al., 2024), which enables models to interact with the Docker environment via bash commands. We make a minor modification to this scaffolding by truncating each model response after the first bash command (i.e., the first bash “block”) instead of returning an error message. Unless otherwise specified, we sample with a temperature of 0.6 and use each model’s default sampling parameters; the exception is GPT-5 mini, which does not expose a temperature parameter. For reasoning models that support a reasoning-effort parameter, we set this parameter to “medium”. We allow for a maximum generation budget of 250 turns, or \$1 in inference budget.

C. Pricing details

We evaluate some models via API queries and others in our local computing cluster. We log the number of input and output tokens and compute the cost in USD using OpenRouter pricing as of January 2025. We use a 90% price reduction for cached inputs, in line with the pricing policies of the OpenAI, Claude, and DeepSeek API platforms. Table 1 summarizes the price-per-token values used.

For mini-coder 4B, we adopt the pricing of Gemma 3 4B, a similarly sized model. Because no models comparable in size to mini-coder 1.7B were available on OpenRouter as of January 2025, we estimate its cost as 40% of mini-coder 4B, approximately matching the ratio of their parameter counts.

C.1. Pricing corrections

Early stopping generations. The mini-swe-agent v1 scaffolding expects a single bash command per turn, but models sometimes fail to follow this template. We therefore modify the scaffolding to truncate each model response after the first

Table 1. Model pricing per 1M tokens.

Model	Input (\$)	Output (\$)	Cache Reduction
Claude Sonnet 4.5	3.00	15.00	90%
GPT-5 mini	0.25	2.00	90%
Qwen 3 Coder 480B	0.25	1.00	90%
DeepSeek v3.2 Reasoner	0.28	0.42	90%
Qwen 3 Coder 30B	0.07	0.27	90%
mini-coder 4B	0.02	0.07	90%
mini-coder 1.7B	0.008	0.028	90%

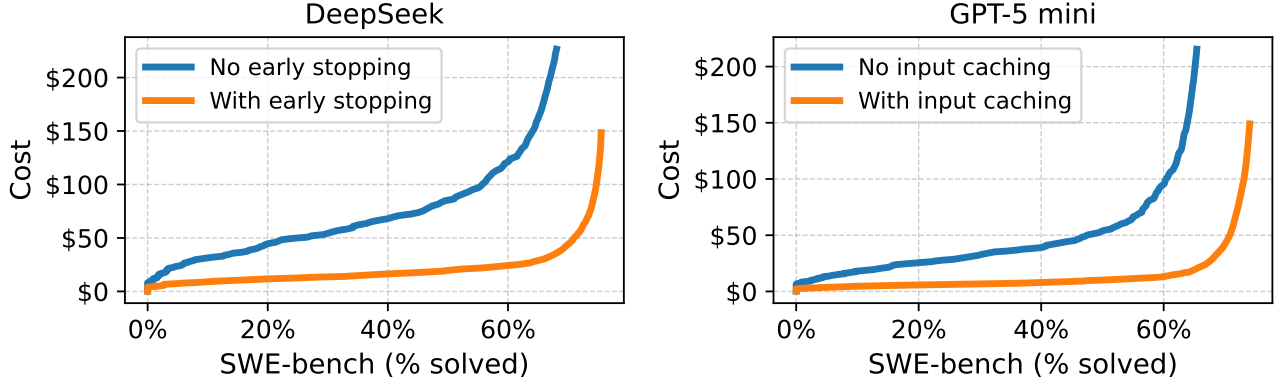


Figure 9. Left: price correction by enforcing early-stopping after the first bash command is generated. Right: price correction by applying the advertised 90% caching discount to GPT-5 mini.

bash command (i.e., the first bash “block”). For simplicity, we allow models to generate their full response and apply this truncation post hoc. However, a more cost-effective approach would enforce the early-stopping criterion during generation. Accordingly, we price trajectories by counting only the tokens up to the first bash block. This adjustment can substantially affect pricing estimates; see DeepSeek V3.2 Reasoner for an example in Figure 9 left.

GPT-5 and caching. We identified a bug in which GPT-5 mini does not cache inputs across turns, resulting in no cost reduction for previously processed context during multi-turn evaluation. We regard this as a bug in the OpenAI API platform rather than intended behavior. To account for this issue, we adjust the trajectory costs by applying the advertised 90% caching discount. Figure 9 right illustrates the impact of the caching bug on the cost–performance trade-off of GPT-5 mini.

D. Lean4 theorem proving

We conduct experiments similar to those in the main text, but for formal theorem proving rather than software issue resolution. Specifically, we consider formal theorem proving using Lean 4 (Moura & Ullrich, 2021), a programming language and proof assistant for formal mathematics. This

domain is verifiable, as the Lean 4 compiler can determine whether a given proof correctly proves a given input statement.

We use the MiniF2F benchmark (Zheng et al., 2022) for evaluation, which includes mathematical problems drawn from high-school, undergraduate, and olympiad exercises. We evaluate the Kimina Prover family of models (Wang et al., 2025a), with model sizes of 0.6B, 1.7B, 8B, and 72B. We scale inference compute via repeated sampling. In contrast to the experiments in the main text, we measure inference budget in floating-point operations (FLOPs), since there are no competitive offerings for the Qwen 3 models on OpenRouter, making comparisons in terms of USD difficult. We approximate inference FLOPs as $C = 2 \times N \times D$ (Hoffmann et al., 2022), where N is the model size and D is the number of generated tokens. We scale inference compute for budgets of up to 3×10^8 FLOPs per input problem.

We next examine the revenue earned by each model in the market. For each performance level between 45% and 75% SWE-bench solve rate, we search for the arbitrage policy with the lowest purchase cost. This cost represents the arbitrage-free (i.e., equilibrium) market price for that level of performance. We plot the corresponding revenue shares in Figure 6 (middle). The arbitrage-free market is not

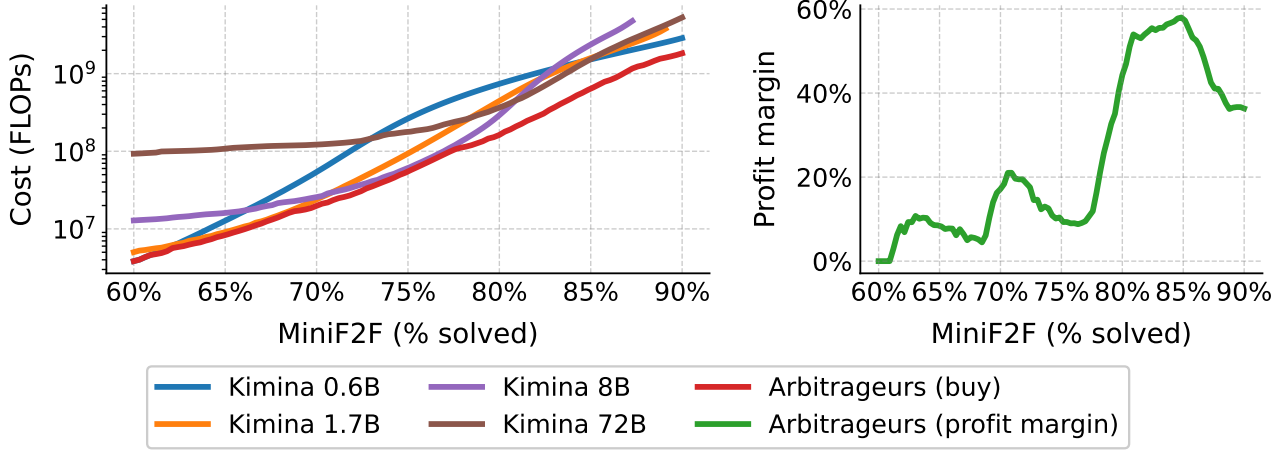


Figure 10. *Left*: Inference cost for each Kimina Prover model to reach various MiniF2F solve rates. We plot in red the best inference cost achievable by an arbitrageur that distributes inference compute across the Kimina models. *Right*: Arbitrageurs can create opportunities for profit, achieving over 60% profit margin.

segmented: four of the six providers share revenue along the performance frontier (e.g., above a 70% SWE-bench solve rate). Two models are uncompetitive and earn no revenue: Claude Sonnet 4.5 is too expensive for the budget regimes considered, while Qwen Coder 480B is neither sufficiently cost-efficient nor high-performing. We plot the cost–performance curves for each of the Kimina models in Figure 10 (left). We then compute arbitrage profitability as follows. For each MiniF2F performance level between 60% and 92% solve rate, we search for the arbitrage policy with the lowest purchase cost, plotted in red. We compute arbitrage profitability by comparing this arbitrage buy cost with the minimum cost for the corresponding performance level across the Kimina models. We plot arbitrage profitability in Figure 10 (right). Arbitrage opportunities exist for all performance levels above a 61% solve rate, with remarkably large profit margins of up to 60%.

We further plot revenue share across providers in Figure 11. In the absence of arbitrageurs, the market is segmented, with a single provider dominating each market segment. In contrast, in the presence of arbitrageurs, providers earn revenue across much broader performance levels. For example, between 78% and 88% solve rate, all four Kimina models earn revenue. As demonstrated earlier, arbitrage profits (or reduction in market prices) come from reductions in providers’ revenue, with the overall marginal revenue of the Kimina models reducing by up to 60%.

D.1. Distillation experiments

We use Kimina Prover 1.7B as the teacher model to reduce the cost of generating training data. We expect that using Kimina 72B for generation would yield stronger arbitrage results. We use Qwen 3 1.7B (Team, 2025) as the

student model. For the seed problems used to generate training trajectories, we use NuminaMath-LEAN (Wang et al., 2025a; LI et al., 2024), which contains 104,000 mathematical competition problems formalized in Lean 4. We sample 8 teacher responses per problem, yielding 832k synthetic responses for NuminaMath-LEAN. We distill models on 68M, 207M, 690M, 2B, and 5.5B tokens. We plot the results in Figure 12. We find that increased distillation consistently improves pass@kk, with larger improvements in pass@100 than in pass@1.

In turn, models distilled on more data dominate those distilled on less data in terms of cost–performance (see Figure 7, middle). Next, we evaluate the arbitrage opportunities enabled by each distilled model when paired with Qwen Coder 72B. We measure mean profitability between 70% and 90% solve rate and plot it against the number of distillation tokens in Figure 7 (right). We observe that increased distillation consistently creates more profitable arbitrage opportunities, with profitability increasing roughly log-linearly with the number of distillation tokens. The model distilled on 5B tokens (400k examples) enables a remarkably high level of profitability, allowing for a profit margin of nearly 30%.

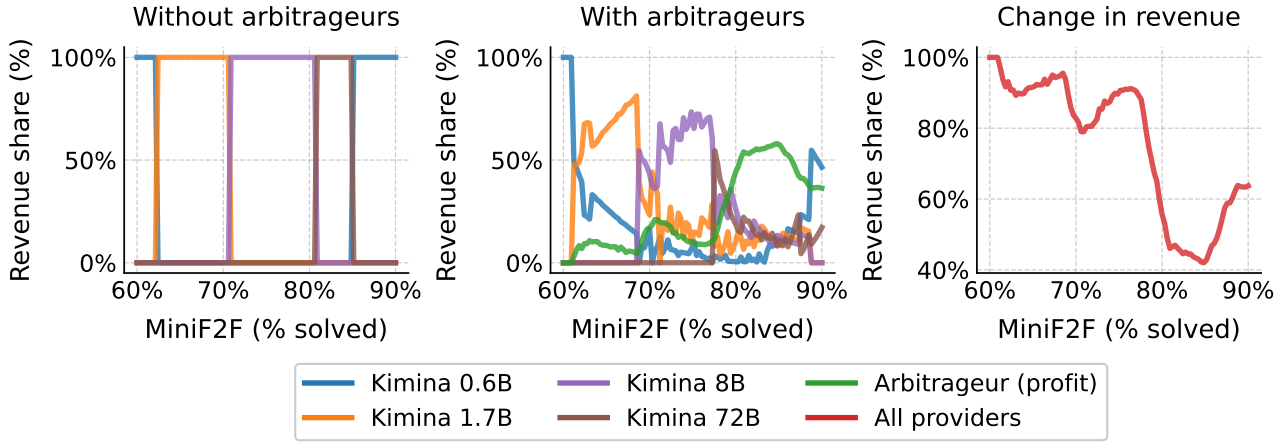


Figure 11. MiniF2F theorem proving. Revenue split across model providers for different levels of performance. *Left:* In the absence of arbitrageurs, the market is segmented by performance, with a single provider dominating each segment. *Middle:* Arbitrageurs eliminate this segmentation, allowing providers to earn revenue across broader ranges of performance. *Right:* Arbitrageurs reduce providers’ marginal revenue.

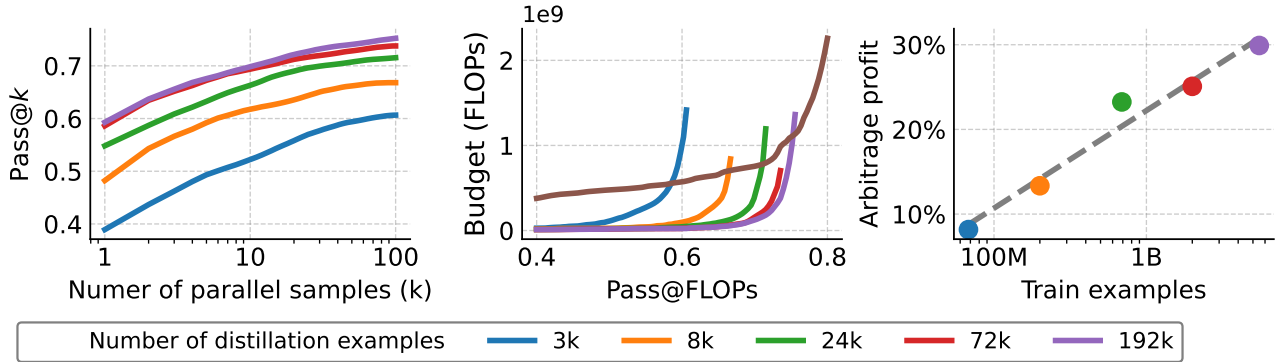


Figure 12. Scaling experiments for Lean theorem proving. We fine-tune Qwen 3 1.7B using synthetic training trajectories from Kimina Prover 1.7B. We distill for up to 200k examples (5B tokens). *Left:* Increased distillation improves test-time scaling. Models distilled on more data Pareto-dominate in terms of pass@k. *Middle:* We explicitly consider inference budget in FLOPs (i.e., pass@FLOPs). We similarly find that increased distillation monotonically improves pass@FLOPs. *Right:* Arbitrage profit when pairing each of the distilled models with Kimina Prover 72B. Models distilled on more teacher data create increasingly more profitable arbitrage opportunities.